Why Elastica Solves PDEs with Explicit Arithmetic

The Method of Lines, Explicit Integration,
and Sequential Evaluation as a PDE Solution Strategy

Snake-HRL Project — Reference Document

1 The Question

The Cosserat rod model is governed by partial differential equations (PDEs) in two independent variables — arc length s and time t. Yet PyElastica's simulation loop contains no iterative solver, no matrix inversion, and no convergence check. Each substep is a sequence of explicit arithmetic operations: compute tangent vectors, evaluate constitutive laws, assemble forces, update velocities, advance positions. Three natural questions arise:

1. Why is Elastica's method called "explicit"?
2. What is the standard way to solve a PDE system, and which approach qualifies as a "solver" in the traditional sense?
3. How can Elastica solve a PDE system through sequential evaluation — that is, without ever solving an equation?

2 Standard Approaches to Solving PDE Systems

A PDE initial–boundary value problem (IBVP) is a continuous mathematical object. No computer can represent continuous functions directly, so *every* numerical method must discretize the problem. The three dominant discretization families are:

1. **Finite differences (FD):** Replace derivatives with algebraic differences on a grid.
2. **Finite elements (FEM):** Approximate the solution as a linear combination of basis functions and enforce the PDE in a weak (integral) sense.
3. **Spectral methods:** Expand the solution in global basis functions (Fourier, Chebyshev) and enforce the PDE in frequency space.

All three families share a common structure: they convert the continuous PDE into a discrete system (algebraic equations or ODEs) that a computer can evaluate.

2.1 Time-dependent PDEs: space first or time first?

For a PDE with both spatial and temporal derivatives, there are two discretization strategies:

Strategy	Procedure	Result
Method of lines (semi-discrete)	Discretize space first. Leave time continuous.	System of ODEs in time: $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, t)$
Full discretization(Rothe's method)	Discretize time first. Solve spatial BVP at each timestep.	Sequence of elliptic problems: $\mathcal{L}_h \, \mathbf{u}^{n+1} = \mathbf{b}^n$

PyElastica uses the **method of lines**: spatial derivatives are replaced by finite differences on a staggered grid, producing an ODE system that is then integrated in time.

2.2 Which approach is the "typical solver"?

In computational solid mechanics (the field that Cosserat rod simulation belongs to), the dominant paradigm is implicit FEM:

- Discretize space with finite elements.
- Discretize time with an implicit scheme (backward Euler, Newmark, generalized-α).
- At each timestep, solve a nonlinear algebraic system via Newton's method, which requires assembling and factorizing a tangent stiffness matrix.

This is what DisMech does, and it is the approach most engineers mean when they say "PDE solver." The Newton iteration is the core computational engine: it solves an equation (the nonlinear residual $\mathbf{g}(\mathbf{q}_{n+1}) = \mathbf{0}$) at every timestep.

PyElastica departs from this paradigm by using an *explicit* time integrator, which avoids the Newton solve entirely.

3 What Discretization Means and How It Works

A PDE describes how a continuous field $\mathbf{u}(s, t)$ evolves over a continuous domain. A computer cannot store or manipulate continuous functions — it works with finite arrays of numbers. **Discretization** is the process of replacing the infinite-dimensional continuous problem with a finite-dimensional discrete problem that a computer can evaluate.

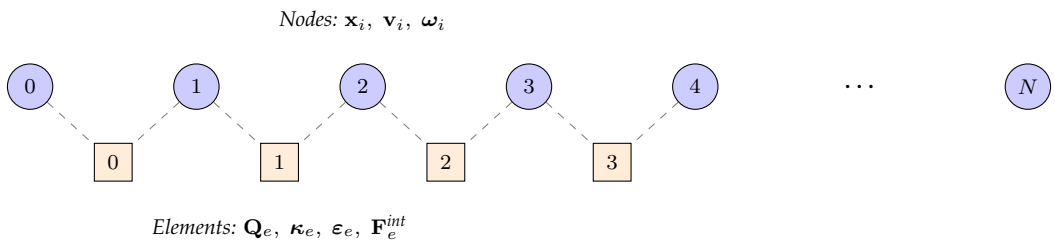
There are two things to discretize: (i) the *domain* (where the solution lives), and (ii) the *operators* (the derivatives that act on the solution). Once both are discretized, the PDE becomes a system of algebraic equations or ODEs in a finite number of unknowns.

3.1 Discretizing the domain: from fields to arrays

The continuous Cosserat rod is parameterized by arc length $s \in [0, L]$. At every point s, the rod carries a position vector $\mathbf{x}(s) \in \mathbb{R}^3$, a velocity $\mathbf{v}(s) \in \mathbb{R}^3$, a director frame $\mathbf{Q}(s) \in \mathbb{R}^{3\times 3}$, and an angular velocity $\boldsymbol{\omega}(s) \in \mathbb{R}^3$. This is an infinite-dimensional object: there are

uncountably many points s, each carrying multiple vector fields.

Spatial discretization replaces this continuum with a *finite set of sample points* (a mesh or grid). For Cosserat rods, the natural choice is a staggered grid:



- **Nodes** ($i = 0, \ldots, N$): carry position $\mathbf{x}_i$, translational velocity $\mathbf{v}_i$, and angular velocity $\boldsymbol{\omega}_i$. These are the *kinematic* quantities — they describe where the rod is and how it moves.
- **Elements** ($e = 0, \ldots, N_e - 1$, connecting adjacent nodes): carry the director frame $\mathbf{Q}_e$, strains $\boldsymbol{\varepsilon}_e$, curvatures $\boldsymbol{\kappa}_e$, and internal forces $\mathbf{F}_e^{\text{int}}$. These are the *constitutive* quantities — they describe how the rod deforms and what forces arise from that deformation.

After this step, the continuous field $\mathbf{u}(s,t)$ is replaced by a finite state vector:

$$\mathbf{s}(t) = \big(\mathbf{x}_0,\ \ldots,\ \mathbf{x}_N,\ \mathbf{v}_0,\ \ldots,\ \mathbf{v}_N,\ \boldsymbol{\omega}_0,\ \ldots,\ \boldsymbol{\omega}_N,\ \mathbf{Q}_0,\ \ldots,\ \mathbf{Q}_{N_e-1}\big)\ \in\ \mathbb{R}^d \tag{1}$$

The infinite-dimensional function space has been replaced by $\mathbb{R}^d$ (here $d = 124$ for the default 21-node rod).

3.2 Discretizing the operators: from derivatives to differences

The PDE contains spatial derivatives like $\partial\mathbf{F}/\partial s$. These operate on the continuous field — they measure how a quantity changes over an infinitesimally small interval ds. On a discrete grid, there is no infinitesimal interval; the smallest distance is the element length $\Delta s = \ell_e$.

A **finite difference** approximates the derivative by a ratio of differences between neighboring grid values:

$$\left.\frac{\partial\mathbf{F}}{\partial s}\right|_{s_i} \approx \frac{\mathbf{F}_{e_i} - \mathbf{F}_{e_{i-1}}}{\Delta s} = \big[\mathbf{D}^T\mathbf{F}_e\big]_i \tag{2}$$

This is the simplest case (central difference on the staggered grid), but the principle is general: *every spatial derivative becomes a weighted sum of neighboring values*. In matrix notation, the derivative operator $\partial/\partial s$ becomes a sparse matrix $\mathbf{D}$ or $\mathbf{D}^T$, and applying the derivative becomes a matrix–vector product.

The key difference operators for the Cosserat rod are:

Continuous operator	Discrete operator	Structure
$\partial/\partial s$ (node $\to$ element)	$\mathbf{D}$	Bidiagonal: $D_{e,i} = \delta_{e,i+1} - \delta_{e,i}$
$\partial/\partial s$ (element $\to$ node)	$\mathbf{D}^T$	Transpose of $\mathbf{D}$
Average (node $\to$ Voronoi)	$\mathbf{A}_v$	Tridiagonal: weighted mean of neighbors
Difference (node $\to$ Voronoi)	$\mathbf{D}_v$	Bidiagonal on Voronoi cells

These matrices are extremely sparse — each row has at most two or three nonzero entries. A matrix–vector product $\mathbf{D}^T\mathbf{F}_e$ involves only $O(N)$ additions and multiplications, not a system of equations.

3.3 Worked example: from PDE to arithmetic on five nodes

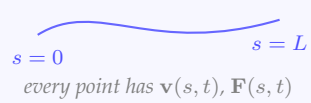
To make the discretization concrete, consider a short rod segment with five nodes ($N = 4$, $N_e = 4$) and uniform element length Δs. We trace how the continuous linear momentum PDE becomes a sequence of arithmetic operations.

Step 0: The continuous PDE. The linear momentum balance for the Cosserat rod is:

$$\rho A \frac{\partial \mathbf{v}}{\partial t} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}} \tag{3}$$

This says: the acceleration at every point s equals the spatial gradient of internal force plus external force. Both sides are continuous functions of s and t.

Step 0 — What we have

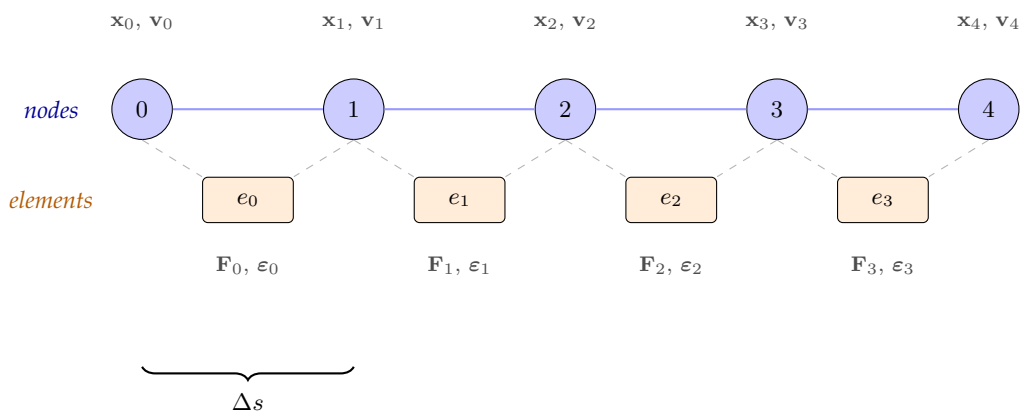


Domain: The rod is a continuous curve parameterized by arc length $s \in [0, L]$.

Unknowns: $\mathbf{v}(s,t)$ and $\mathbf{F}(s,t)$ are *fields* — they assign a vector to every point s on the rod, at every instant t. There are *uncountably many* values.

Derivatives: $\partial\mathbf{F}/\partial s$ is a true derivative — a limit of $(\mathbf{F}(s+h) - \mathbf{F}(s))/h$ as $h \to 0$.

Step 1: Discretize the domain. Replace the continuous rod with five nodes at positions s_0, s_1, s_2, s_3, s_4 and four elements connecting them:

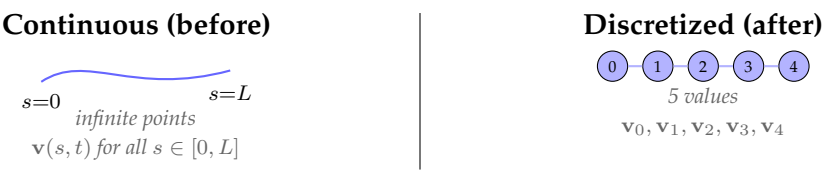


The continuous velocity field $\mathbf{v}(s,t)$ becomes five velocity vectors:

$$\mathbf{v}(s,t) \longrightarrow \mathbf{v}_0(t),\ \mathbf{v}_1(t),\ \mathbf{v}_2(t),\ \mathbf{v}_3(t),\ \mathbf{v}_4(t)$$

Similarly, the internal force field becomes four element force vectors $\mathbf{F}_0, \mathbf{F}_1, \mathbf{F}_2, \mathbf{F}_3$ (one per element).

Step 1 — Continuous vs. discretized domain



What changed: The continuous field $\mathbf{v}(s,t)$ defined at uncountably many points was replaced by a finite list of 5 vectors. The rod's state went from an infinite-dimensional function to a finite-dimensional vector $\mathbf{s} \in \mathbb{R}^{124}$. No information is lost about these 5 points; the approximation is that we *ignore* what happens between them.

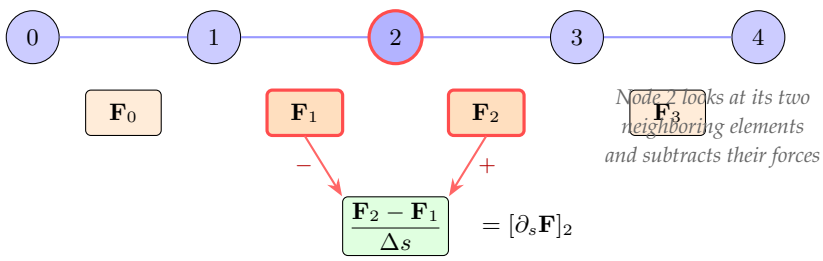
Step 2: Discretize the spatial derivative. The continuous derivative $\partial \mathbf{F}/\partial s$ is replaced by finite differences. For an interior node i, the force gradient is approximated by the difference of the forces in the two adjacent elements:

$$\left.\frac{\partial \mathbf{F}}{\partial s}\right|_{s_i} \approx \frac{\mathbf{F}_{e_i} - \mathbf{F}_{e_{i-1}}}{\Delta s} \tag{4}$$

Writing this out for every node (with boundary treatment at the endpoints):

$$\underbrace{\begin{pmatrix} [\partial_s \mathbf{F}]_0 \\ [\partial_s \mathbf{F}]_1 \\ [\partial_s \mathbf{F}]_2 \\ [\partial_s \mathbf{F}]_3 \\ [\partial_s \mathbf{F}]_4 \end{pmatrix}}_{\text{nodal force gradients}} = \frac{1}{\Delta s} \underbrace{\begin{pmatrix} -1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\mathbf{D}^T\ (5\times 4)} \underbrace{\begin{pmatrix} \mathbf{F}_0 \\ \mathbf{F}_1 \\ \mathbf{F}_2 \\ \mathbf{F}_3 \end{pmatrix}}_{\text{element forces}} \tag{5}$$

The following diagram shows what this matrix–vector product does for node 2:



The continuous derivative $\partial \mathbf{F}/\partial s$ became a *matrix–vector multiply*. For node 2, the result is simply $(\mathbf{F}_2 - \mathbf{F}_1)/\Delta s$ — the difference of two neighboring values, divided by the spacing. This is **arithmetic**, not an equation to solve.

Step 2 — Continuous vs. discretized spatial derivative

Continuous (before)	**Discretized (after)**
$\left.\frac{\partial \mathbf{F}}{\partial s}\right\|_{s_2} = \lim_{h\to 0} \frac{\mathbf{F}(s_2+h) - \mathbf{F}(s_2)}{h}$	$\left.\frac{\partial \mathbf{F}}{\partial s}\right\|_{s_2} \approx \frac{\mathbf{F}_2 - \mathbf{F}_1}{\Delta s}$
Requires knowing $\mathbf{F}$ at *infinitesimally close* points. This is a limit — cannot be computed on a discrete grid.	Uses only two neighboring force values that we already have. This is a subtraction followed by a division.

What changed: The infinitesimal limit $h \to 0$ was replaced by a *finite* difference with step size Δs. The approximation improves as Δs shrinks (more nodes), with error proportional to $(\Delta s)^2$ for a centered scheme. The key point: the derivative, which is defined as a limit and cannot be computed exactly on a computer, became a subtraction of two known numbers.

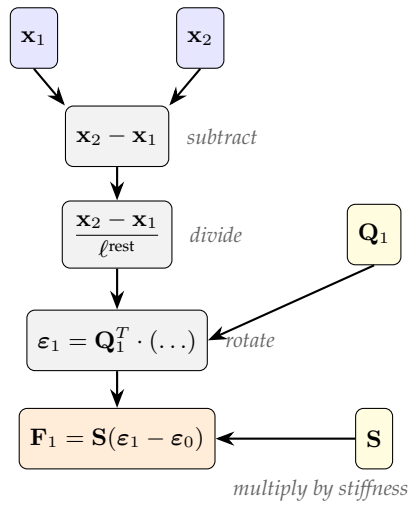
Step 3: Compute element forces from the constitutive law. The element forces $\mathbf{F}_e$ are not free variables — they are determined by the current deformation. The constitutive law relates force to strain:

$$\mathbf{F}_e = \mathbf{S}\left(\boldsymbol{\varepsilon}_e - \boldsymbol{\varepsilon}_e^{\text{rest}}\right) \tag{6}$$

where $\mathbf{S} = \mathrm{diag}(GA,\, GA,\, EA)$ is the stiffness matrix (shear, shear, stretch), and the strain is computed from node positions:

$$\boldsymbol{\varepsilon}_e = \mathbf{Q}_e^T \frac{\mathbf{x}_{e+1} - \mathbf{x}_e}{\ell_e^{\text{rest}}} \tag{7}$$

Again, this is pure arithmetic: subtract two position vectors, divide by rest length, rotate into the local frame, multiply by stiffness. The following diagram traces the computation for element 1:



Every box is computed from the boxes above it using basic operations (subtract, divide, rotate, multiply). The yellow boxes ($\mathbf{Q}_1$, $\mathbf{S}$) are known quantities from the current state and material properties.

Step 3 — Continuous vs. discretized constitutive law

Continuous (before)	**Discretized (after)**
$\mathbf{F}(s,t) = \mathbf{S}\left[\mathbf{Q}(s,t)^T \frac{\partial \mathbf{x}}{\partial s}(s,t) - \boldsymbol{\varepsilon}_0\right]$	$\mathbf{F}_1 = \mathbf{S}\left[\mathbf{Q}_1^T \frac{\mathbf{x}_2 - \mathbf{x}_1}{\ell^{\text{rest}}} - \boldsymbol{\varepsilon}_0\right]$
The strain $\partial\mathbf{x}/\partial s$ is again a spatial derivative — a limit. $\mathbf{Q}(s,t)$ is a rotation field defined at every point. This requires the continuous tangent vector everywhere.	The spatial derivative $\partial\mathbf{x}/\partial s$ was replaced by $(\mathbf{x}_2 - \mathbf{x}_1)/\ell^{\text{rest}}$ — a finite difference between two known position vectors.

What changed: The constitutive law itself is the same algebraic formula — it was never a differential equation. The discretization enters through the *strain* computation: the continuous tangent $\partial\mathbf{x}/\partial s$ became a finite difference $(\mathbf{x}_{e+1} - \mathbf{x}_e)/\ell^{\text{rest}}$. This is another instance of replacing a limit with a subtraction.

Step 4: Assemble the acceleration. Substituting the discretized force gradient into the momentum equation gives the acceleration at each node:

$$\dot{\mathbf{v}}_i = \frac{1}{\rho A}\left(\frac{\mathbf{F}_{e_i} - \mathbf{F}_{e_{i-1}}}{\Delta s} + \mathbf{f}_{\text{ext},i}\right) \tag{8}$$

For node 2, this expands to a fully explicit expression in terms of the current positions:

$$\dot{\mathbf{v}}_2 = \frac{1}{\rho A\,\Delta s}\left[\mathbf{S}\left(\mathbf{Q}_2^T\frac{\mathbf{x}_3 - \mathbf{x}_2}{\ell^{\text{rest}}} - \boldsymbol{\varepsilon}_0\right) - \mathbf{S}\left(\mathbf{Q}_1^T\frac{\mathbf{x}_2 - \mathbf{x}_1}{\ell^{\text{rest}}} - \boldsymbol{\varepsilon}_0\right)\right] + \frac{\mathbf{f}_{\text{ext},2}}{\rho A} \tag{9}$$

Every quantity on the right-hand side ($\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{Q}_1, \mathbf{Q}_2$) is part of the *current* state. The acceleration $\dot{\mathbf{v}}_2$ is computed by direct evaluation — no iteration, no unknown on the right-hand side.

Step 4 — Continuous vs. discretized momentum equation

Continuous (before)	**Discretized (after)**
$\rho A\frac{\partial \mathbf{v}}{\partial t}(s,t) = \frac{\partial \mathbf{F}}{\partial s}(s,t) + \mathbf{f}_{\text{ext}}(s,t)$	$\rho A\,\dot{\mathbf{v}}_2 = \frac{\mathbf{F}_2 - \mathbf{F}_1}{\Delta s} + \mathbf{f}_{\text{ext},2}$
This holds at every point s simultaneously — it is a *partial* differential equation coupling space and time. Computing $\partial\mathbf{v}/\partial t$ requires knowing the force gradient at all s.	This is one equation for one node, using two known force values. It is an *ordinary* differential equation — the only derivative left is d/dt.

What changed: By replacing $\partial\mathbf{F}/\partial s$ with a finite difference, the spatial coupling disappeared. Each node's acceleration depends only on its immediate neighbors' forces (which are already computed in Step 3). The PDE became a system of ODEs — one per node, each involving only time derivatives.

Step 5: Advance in time (explicit Verlet). With the acceleration known, the Verlet integrator updates the velocity and position:

$$\mathbf{x}_2^{n+1/2} = \mathbf{x}_2^n + \tfrac{1}{2}\Delta t\;\mathbf{v}_2^n \tag{10}$$

$$\mathbf{v}_2^{n+1} = \mathbf{v}_2^n + \Delta t\;\dot{\mathbf{v}}_2(\mathbf{x}^{n+1/2}) \tag{11}$$

$$\mathbf{x}_2^{n+1} = \mathbf{x}_2^{n+1/2} + \tfrac{1}{2}\Delta t\;\mathbf{v}_2^{n+1} \tag{12}$$

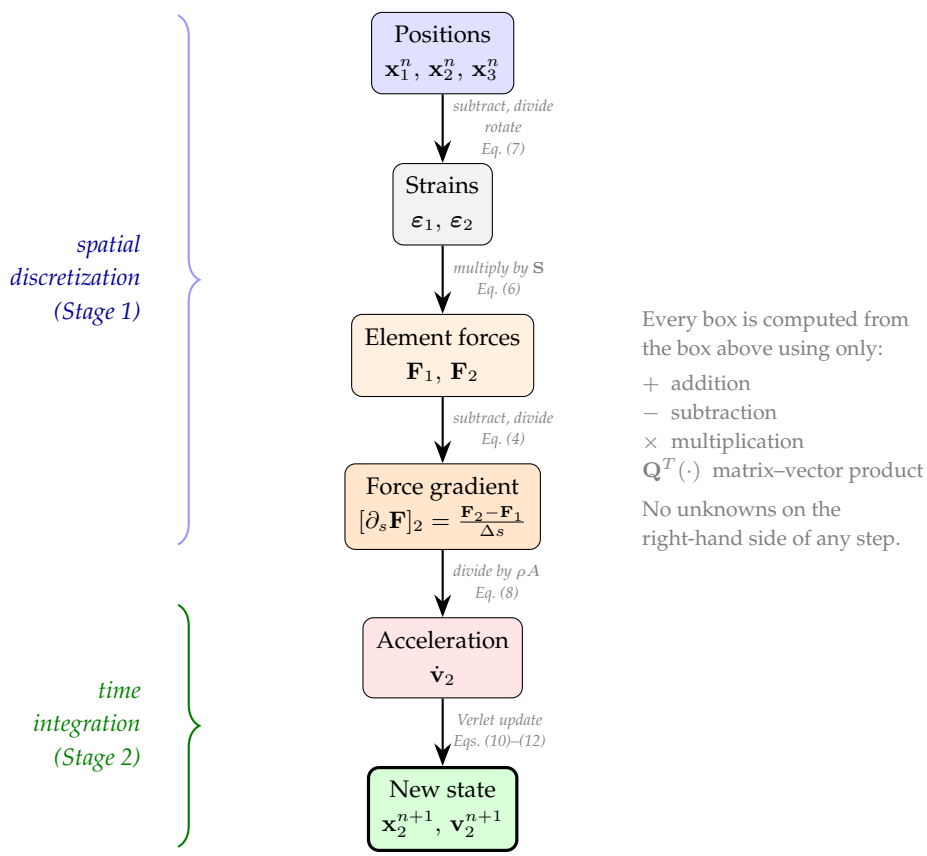
Each line uses only quantities that are already known. The entire update for one node is: 6 subtractions, 4 matrix–vector multiplies, and 3 additions — roughly 50 floating-point operations per node per substep.

Step 5 — Continuous vs. discretized time evolution

Continuous (before)	**Discretized (after)**
$$\mathbf{v}_2(t+\Delta t) = \mathbf{v}_2(t) + \int_t^{t+\Delta t} \dot{\mathbf{v}}_2(\tau)\, d\tau$$	$$\mathbf{v}_2^{n+1} = \mathbf{v}_2^n + \Delta t\, \dot{\mathbf{v}}_2^{n+1/2}$$
The exact update requires integrating the acceleration over the interval $[t, t{+}\Delta t]$. This integral requires knowing $\dot{\mathbf{v}}_2(\tau)$ at *every* instant in the interval — not just the endpoints.	The integral was replaced by a rectangle: Δt times the acceleration at the midpoint. This is one multiplication and one addition — pure arithmetic.

What changed: The continuous time integral (which requires knowing the acceleration for all τ) became a single multiply-and-add using the acceleration at just one known time. The approximation improves as Δt shrinks (more substeps), with Verlet giving second-order accuracy: error $\propto (\Delta t)^2$. This is *time discretization* — the second and final stage. After this step, no continuous quantities remain: both space *and* time have been replaced by finite grids, and every operation is arithmetic.

The complete chain. Tracing through one substep for node 2, the continuous PDE has been reduced to the following pipeline of arithmetic operations:



Every arrow is addition, subtraction, multiplication, or a matrix–vector product. At no point does an unknown quantity appear on the right-hand side of any expression. This is what it means for a PDE to be "solved" by explicit evaluation: the continuous derivatives have been replaced by arithmetic on neighboring grid values, and the time derivative has been replaced by a forward step from the known state.

3.4 The result: PDE becomes a system of ODEs

After discretizing the domain and the spatial operators, all spatial dependence has been eliminated. What remains is a system of ordinary differential equations in time:

$$\dot{\mathbf{s}}(t) = \mathbf{g}\big(\mathbf{s}(t),\, \mathbf{a}(t)\big) \tag{13}$$

where $\mathbf{g} : \mathbb{R}^d \to \mathbb{R}^d$ is a *known, closed-form function* that encodes the entire spatial physics:

$$\mathbf{g}(\mathbf{s}) = \underbrace{\text{geometry}}_{\mathbf{x}_i \to \mathbf{t}_e, \ell_e} \;\to\; \underbrace{\text{strains}}_{\mathbf{t}_e \to \varepsilon_e, \kappa_e} \;\to\; \underbrace{\text{constitutive law}}_{\varepsilon_e \to \mathbf{F}_e, \kappa_e \to \mathbf{M}_e} \;\to\; \underbrace{\text{force balance}}_{\mathbf{D}^T\mathbf{F}_e + \mathbf{f}_{\text{ext}}} \;\to\; \underbrace{\text{accelerations}}_{\mathbf{F}_{\text{net}}/(\rho A)} \tag{14}$$

Every step in this pipeline is an explicit algebraic formula — multiplication, addition, cross products, matrix–vector products. No step requires solving an equation. This is why the method-of-lines approach converts the PDE into something that can be "solved" by sequential evaluation.

3.5 Comparison: how the three discretization families handle this

Different discretization families (Section 2) differ in how they replace the spatial derivatives, but the end result is always the same — a finite system that a computer can evaluate:

Family	**How it discretizes space**	**What the PDE becomes**
Finite differences	Replace $\partial/\partial s$ with ratios of neighbor values on a grid	System of ODEs (if time left continuous) or algebraic equations (if time also discretized)
Finite elements	Approximate $\mathbf{u}(s) \approx \sum_j c_j \phi_j(s)$ with basis functions; enforce PDE via weighted integrals	$\mathbf{M}\,\dot{\mathbf{c}} = \mathbf{f}(\mathbf{c})$ (mass matrix times rate = force vector)
Spectral methods	Expand $\mathbf{u}(s) = \sum_k \hat{u}_k\, e^{ik\pi s/L}$; operate in frequency space	$\dot{\hat{\mathbf{u}}} = \mathbf{f}(\hat{\mathbf{u}})$ (ODEs for spectral coefficients)

PyElastica uses the first approach (finite differences on a staggered grid) because Cosserat rods are one-dimensional in space, making finite differences natural and efficient. For more complex geometries (2D/3D), finite elements are preferred because they can handle irregular meshes.

4 Why Elastica's Method Is Called "Explicit"

The terms "explicit" and "implicit" refer to how the time integrator computes the next state.

4.1 Definitions

Consider an ODE system $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, t)$ with current state $\mathbf{s}^n$ at time t^n.

Explicit method. The next state is computed directly from the current state:

$$\mathbf{s}^{n+1} = \Phi(\mathbf{s}^n, t^n, \Delta t) \tag{15}$$

where Φ is a known, closed-form function. The unknown $\mathbf{s}^{n+1}$ appears *only on the left-hand side*. Given $\mathbf{s}^n$, you *evaluate* Φ to obtain $\mathbf{s}^{n+1}$. No equation is solved.

Implicit method. The next state satisfies an equation that involves itself:

$$\mathbf{s}^{n+1} = \Psi(\mathbf{s}^{n+1}, \mathbf{s}^n, t^n, \Delta t) \tag{16}$$

The unknown $\mathbf{s}^{n+1}$ appears *on both sides*. You cannot simply evaluate the right-hand side — you must *solve* for $\mathbf{s}^{n+1}$, typically via Newton iteration.

4.2 Concrete examples

Method	**Update rule**	**Type**
Forward Euler	$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^n)$	Explicit
Backward Euler	$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^{n+1})$	Implicit
Position Verlet	Eqs. (17a)–(17c)	Explicit

4.3 Position Verlet is explicit

The Position Verlet scheme used by PyElastica consists of three substeps:

$$\mathbf{q}^{n+\frac{1}{2}} = \mathbf{q}^n + \tfrac{1}{2}\,\Delta t\;\dot{\mathbf{q}}^n \tag{17a}$$

$$\dot{\mathbf{q}}^{n+1} = \dot{\mathbf{q}}^n + \Delta t\;\ddot{\mathbf{q}}\Big(\mathbf{q}^{n+\frac{1}{2}}\Big) \tag{17b}$$

$$\mathbf{q}^{n+1} = \mathbf{q}^{n+\frac{1}{2}} + \tfrac{1}{2}\,\Delta t\;\dot{\mathbf{q}}^{n+1} \tag{17c}$$

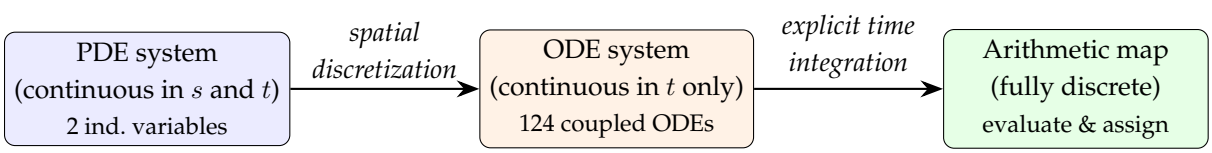
Every quantity on the right-hand side of each equation is *already known* when that equation is evaluated:

- Eq. (17a): uses $\mathbf{q}^n$ and $\dot{\mathbf{q}}^n$ (the current state).
- Eq. (17b): uses $\dot{\mathbf{q}}^n$ and $\ddot{\mathbf{q}}(\mathbf{q}^{n+1/2})$. The acceleration is computed from $\mathbf{q}^{n+1/2}$, which was just determined in the previous line.
- Eq. (17c): uses $\mathbf{q}^{n+1/2}$ and $\dot{\mathbf{q}}^{n+1}$, both already computed.

At no point does an unknown appear on both sides of an equation. The entire substep is a **sequential evaluation** — a chain of assignments, not a system of equations to solve.

5 How Elastica Solves a PDE Through Sequential Evaluation

The key insight is a two-stage reduction:



5.1 Stage 1: Spatial discretization eliminates the spatial variable

The Cosserat rod PDE contains two types of derivatives: $\partial/\partial s$ (spatial) and $\partial/\partial t$ (temporal). Spatial discretization replaces all $\partial/\partial s$ derivatives with finite difference operators on the staggered grid ($N = 21$ nodes, $N_e = 20$ elements).

What the spatial derivatives become. Each continuous spatial derivative maps to a matrix–vector product:

Continuous	Discrete	Operation
$\dfrac{\partial \mathbf{F}}{\partial s}$	$\mathbf{D}^T \mathbf{F}_e$	Bidiagonal matrix $\times$ element force vector
$\dfrac{\partial \mathbf{M}}{\partial s}$	$\mathbf{D}_v^T \boldsymbol{\tau}_v$	Voronoi difference matrix $\times$ moment vector
$\boldsymbol{\kappa}(s) - \boldsymbol{\kappa}_0(s)$	$\kappa_e - \kappa_e^{\text{rest}}$	Element-wise subtraction
$\boldsymbol{\varepsilon}(s)$	$\mathbf{Q}_e^T \, \Delta \mathbf{x}_e / \ell_e^{\text{rest}}$	Rotation $\times$ finite difference

After this replacement, no spatial derivative remains. The continuous PDE in (s, t) has become a system of 124 coupled ODEs in t alone:

$$\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s},\, \mathbf{a},\, t), \qquad \mathbf{s} \in \mathbb{R}^{124} \tag{18}$$

The function $\mathbf{g}$ encodes all the spatial coupling: each node's acceleration depends on its neighbors through the difference operators $\mathbf{D}$ and $\mathbf{D}^T$. But $\mathbf{g}$ is a *closed-form function* — given the current state $\mathbf{s}$, you can compute $\mathbf{g}(\mathbf{s})$ by evaluating a sequence of explicit formulas (geometry $\to$ strains $\to$ constitutive law $\to$ forces $\to$ accelerations).

Why this works. The difference matrices ($\mathbf{D}$, $\mathbf{D}^T$, $\mathbf{A}_v$, $\mathbf{D}_v$) are sparse, bidiagonal or tridiagonal. A matrix–vector product $\mathbf{D}^T \mathbf{F}_e$ is just arithmetic: each entry is a weighted sum of two neighboring values. The constitutive law $\mathbf{F} = \mathbf{S}(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_0)$ is an algebraic relation: multiply stiffness by strain. None of these operations requires solving a system of equations.

5.2 Stage 2: Explicit time integration eliminates the temporal variable

With the ODE system (18) in hand, an explicit integrator advances the state by one substep using only the *current* (known) state:

$$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t \cdot \mathbf{h}(\mathbf{s}^n) \tag{19}$$

where $\mathbf{h}$ involves evaluating $\mathbf{g}$ (for Verlet, at the half-step configuration). Since $\mathbf{g}$ is a closed-form function and $\mathbf{s}^n$ is known, the entire right-hand side is computable by direct evaluation.

Contrast with implicit methods. An implicit integrator (e.g., backward Euler) writes:

$$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t \cdot \mathbf{g}(\mathbf{s}^{n+1}) \tag{20}$$

Now $\mathbf{s}^{n+1}$ appears on *both sides*. This is a system of 124 nonlinear equations in 124 unknowns, requiring Newton iteration to solve. Each Newton step assembles the Jacobian

$\partial \mathbf{g}/\partial \mathbf{s}$ and solves a linear system — this is what makes implicit methods expensive per step.

5.3 The complete picture: Elastica's substep as a function composition

A single Verlet substep in Elastica is a composition of explicit operations:

$$V_{\Delta t}: \quad \mathbf{s}^n \xrightarrow{\text{half-step}} \mathbf{q}^{n+1/2} \xrightarrow{\text{geometry}} (\mathbf{t}_e, \ell_e, \varepsilon_e) \xrightarrow{\text{constitutive}} (\mathbf{F}^{\text{int}}, \mathbf{M}^{\text{int}}) \xrightarrow{\text{forces}} \ddot{\mathbf{q}} \xrightarrow{\text{update}} \mathbf{s}^{n+1} \tag{21}$$

Every arrow is a *function evaluation*, not an equation to solve. The entire pipeline is a deterministic map $\mathbb{R}^{124} \to \mathbb{R}^{124}$ computed in a single forward pass — no iteration, no convergence check, no matrix inversion.

One RL control step composes $n_{\text{sub}} = 500$ of these maps:

$$T(\mathbf{s}_t, \mathbf{a}_t) = \underbrace{V_{\Delta t} \circ \cdots \circ V_{\Delta t}}_{500}(\mathbf{s}_t;\, \mathbf{a}_t) \tag{22}$$

6 Why Explicit Methods Need Small Timesteps

The price of avoiding an equation solve is a **stability constraint**. Explicit methods are conditionally stable: if Δt exceeds a critical value, the numerical solution grows unboundedly.

For the Cosserat rod, the critical timestep is set by the fastest wave (bending waves in the stiffest mode):

$$\Delta t < C \sqrt{\frac{\rho A\, \Delta s^4}{EI}} \tag{23}$$

where $\Delta s = L/N_e$ is the element length and C is an order-unity constant. With the project's default parameters, this gives $\Delta t_{\max} \approx 1.7 \times 10^{-3}$ s, and Elastica uses $\Delta t = 10^{-3}$ s (safely below the limit).

Implicit methods (backward Euler, Newmark) are *unconditionally stable*: any Δt produces a bounded solution. This is why DisMech can use $\Delta t = 0.05$ s (50× larger), but pays for it with a Newton solve at every step.

Table 1: The explicit–implicit tradeoff.

	Explicit (Elastica)	**Implicit (DisMech)**
Per-step cost	Low (evaluate formulas)	High (Jacobian + linear solve)
Step size	Small ($\Delta t = 0.001\,\mathrm{s}$)	Large ($\Delta t = 0.05\,\mathrm{s}$)
Steps per RL action	500	1
Stability	Conditional (CFL limit)	Unconditional
Solves an equation?	No	Yes (Newton iteration)
"Solver" in the classical sense?	No	Yes

7 Implications for the Neural Surrogate

The neural surrogate f_θ replaces the 500-step composition T with a single forward pass. Understanding *what it replaces* clarifies the surrogate's role:

1. **It does not replace a PDE solver.** Elastica was never solving the PDE in the traditional sense (no Newton iteration, no stiffness matrix). It was *evaluating* 500 sequential maps.

2. **It replaces a function composition.** The ground truth is $T = V_{\Delta t}^{500}$, a composition of 500 explicit maps. The surrogate f_θ approximates this composition in a single evaluation:
$$f_\theta(\mathbf{s}_t,\, \mathbf{a}_t) \approx T(\mathbf{s}_t,\, \mathbf{a}_t) - \mathbf{s}_t \tag{24}$$

3. **The "difficulty" is compositional, not algebraic.** Each individual Verlet substep is cheap and simple. What makes the transition operator expensive is that 500 substeps must be composed sequentially — the output of each feeds into the next. The surrogate's value is collapsing this sequential composition into a single parallel computation (a neural network forward pass).

8 Summary

1. **"Explicit" means the next state is computed by evaluation, not by solving an equation.** In Position Verlet, every right-hand side is already known when it is evaluated. The unknown $\mathbf{s}^{n+1}$ never appears on the right-hand side.

2. **The standard PDE solver uses implicit time integration with Newton iteration.** This is what DisMech does and what most engineers mean by "solving a PDE." The Newton solve — assembling the Jacobian and factorizing a sparse linear system — is the computational bottleneck.

3. **Elastica solves a PDE through the method of lines plus explicit integration.** Spatial discretization converts $\partial/\partial s$ into matrix–vector products (Stage 1). Explicit time integration converts $\partial/\partial t$ into sequential evaluation (Stage 2). After both stages, the entire PDE simulation is reduced to a chain of arithmetic operations — no equation is ever solved. The price is a CFL stability constraint that forces small timesteps.

The neural surrogate replaces this chain of 500 arithmetic maps with a single forward pass, trading the CFL-enforced sequential composition for a learned parallel computation.

References

[1] J. C. Strikwerda, *Finite Difference Schemes and Partial Differential Equations*, 2nd ed., SIAM, 2004.

[2] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, 2007.

[3] E. Hairer, C. Lubich, and G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, 2nd ed., Springer, 2006.

[4] M. Gazzola, L. H. Dudte, A. G. McCormick, and L. Mahadevan, "Forward and inverse problems in the mechanics of soft filaments," *Royal Society Open Science*, vol. 5, no. 6, 2018.

[5] M. Bergou, M. Wardetzky, S. Robinson, B. Audoly, and E. Grinspun, "Discrete elastic rods," *ACM Transactions on Graphics (SIGGRAPH)*, vol. 27, no. 3, 2008.